

쿠버네티스 환경에서의 PaC 위반 대응 및 예외 관리 시스템 설계



부산대학교 정보컴퓨터공학부

팀명 : 팩

지도교수 : 유영환

yeongrimgo1106@pusan.ac.kr 202355702 고영림

pty4pp@pusan.ac.kr 202155559 박태영

yuyeol@pusan.ac.kr 202148124 최유열

목차

1. 기술적 배경
2. 문제 및 목표 정의
 - 2.1 정책 위반 피드백의 모호성
 - 2.2 정책 예외 처리 관리 및 추적의 어려움
 - 2.3 기존 도구와의 차별성
 - 2.4 목표 정의
3. 요구사항 분석 및 시스템 구성
 - 3.1 요구사항 분석
 - 3.1.1 기능적 요구사항
 - 3.1.2 비기능적 요구사항
 - 3.2 시스템 구성
4. 제약사항 및 기술적 어려움
 - 4.1 오류 메시지 해석의 정확성
 - 4.2 관리자 인증 및 service account 권한 관리
 - 4.3 만료 처리와 이력 보존의 분리
 - 4.4 데이터 저장 비용과 성능 문제
5. 구현 계획 및 역할 분배
 - 5.1 구현 계획
 - 5.2 역할 분배

1. 기술적 배경

최근 클라우드 네이티브 환경에서는 Kubernetes 기반 애플리케이션 배포가 보편화되고 있다. Kubernetes는 컨테이너 기반 애플리케이션을 효율적으로 배포하고 운영할 수 있는 환경을 제공하지만, 잘못된 리소스 설정이나 보안 기준 위반이 그대로 배포될 경우 운영 장애나 보안 취약점으로 이어질 수 있다.

이러한 문제를 방지하기 위해 Policy-as-Code, 이하 PaC 방식의 정책 엔진을 활용하는 사례가 증가하고 있다. PaC는 보안 정책이나 운영 정책을 코드 형태로 정의하고, 배포 과정 또는 Kubernetes admission 단계에서 자동으로 검증하는 방식이다. 대표적인 Kubernetes 정책 엔진으로는 Kyverno, OPA Gatekeeper 등이 있으며, 본 프로젝트에서는 Kubernetes-native 방식으로 정책을 정의할 수 있는 Kyverno를 중심으로 시스템을 설계한다.

Kyverno는 Kubernetes 리소스가 생성되거나 수정될 때 정책을 평가하고, 정책 위반이 발생하면 해당 요청을 차단하거나 감사 결과를 남길 수 있다. 또한 PolicyException 리소스를 통해 특정 정책과 rule에 대해 제한적인 예외를 설정할 수 있다.

그러나 PaC는 단순한 로그 생성이 아니라 실제 배포를 차단하므로, 정책 위반은 추후 감사 대상이 아니라 개발자의 업무 흐름을 직접 중단시키는 문제가 된다. 이때 개발자는 어떤 정책이 왜 실패했는지, 어떤 설정을 수정해야 하는지, 예외가 필요한 경우 누구에게 어떻게 요청해야 하는지 파악해야 한다.

실제 조직에서는 이러한 정책 예외 처리 과정이 여러 협업 채널에 분산되기 쉬우므로, 정책 엔진의 집행 기능만으로는 위반 이후의 대응과 예외 관리를 충분히 지원하기 어렵다. 따라서 본 프로젝트는 정책 위반 이후의 대응 흐름과 예외 관리 절차를 통합적으로 지원하는 보조 시스템 설계를 목표로 한다.

2. 문제 및 목표 정의

2.1 정책 위반 피드백의 모호성

다양한 조건을 복합적으로 적용하는 Kubernetes 환경에서 정책 위반이 발생하면 정책 엔진이 반환하는 메시지는 정책명이나 rule명 중심이다. 이 경우 어떤 설정이 문제인지 파악하기 위해서는 개발자는 admission violation, CI 로그, Kubernetes Event, PolicyReport 등 여러 위치에서 정보를 확인해야 할 수 있다. 이는 인지 부담 증가와 디버깅 비용 증가로 이어진다.

2.2 정책 예외 처리 관리 및 추적의 어려움

운영 환경에서는 긴급 장애 대응, 테스트, 마이그레이션 등으로 인해 일시적인 정책 예외가 필요한 경우가 있다. Kyverno는 PolicyException 리소스를 통해 정책 예외를 표현할 수 있다.

그러나 실제 예외 요청과 승인 과정은 Slack, Jira, 이메일, GitHub Pull Request 등 여러 채널에서 이루어질 수 있다. 이 경우 누가 예외를 요청했는지, 어떤 사유로 승인되었는지, 어떤 리소스에 적용되었는지, 언제 만료되어야 하는지 추적하기 어렵다.

2.3 기존 도구와의 차별성

Kyverno는 정책 집행, PolicyReport, PolicyException, cleanup 등 강력한 기능을 제공한다. 또한 Prometheus와 Grafana 같은 관측 도구를 활용하면 정책 관련 metric을 시각화할 수 있다.

그러나 기존 도구들은 주로 정책 평가 결과나 시스템 상태를 보여주는 데 초점을 둔다. 반면 본 프로젝트는 정책 위반 이후 개발자와 관리자가 수행해야 하는 업무 흐름에 초점을 둔다.

기존 기능	주요 역할	한계
Kyverno 정책	리소스 검증 및 차단	정책 위반 이후의 예외 요청·승인 흐름은 별도로 구성해야 함
PolicyReport	정책 평가 결과 기록	예외 요청자, 승인자, 만료 이력 등 거버넌스 정보는 부족함
PolicyException	정책 예외 적용	요청·승인·감사 이력 관리는 별도 시스템 필요
Grafana / Prometheus	metric 시각화 및 알림	예외 요청과 승인 같은 업무 흐름 처리에는 한계가 있음
Slack / Jira / 이메일	의사소통	기록이 분산되어 사후 감사와 만료 관리가 어려움

본 프로젝트는 기존 도구를 대체하지 않고, 그 위에 정책 위반 대응과 예외 거버넌스를 위한 통합 워크플로우를 추가하는 것을 차별점으로 한다.

2.4 목표 정의

본 프로젝트의 목적은 Kubernetes 환경에서 PaC 정책 위반이 발생했을 때 개발자가 원인과 수정 방향을 쉽게 파악할 수 있도록 돕고, 임시 정책 예외의 요청·승인·만료·이력 관리를 하나의 시스템에서 수행할 수 있도록 하는 것이다.

구체적인 목표는 다음과 같다.

1. 파편화된 정책 위반 정보의 통합 및 가시성 확보
2. 통합된 예외 처리 워크플로우 구축
3. 예외의 생명주기 자동화 및 이력 보존
4. 관리자 의사결정 지원

3. 요구사항 분석 및 시스템 구성

3.1 요구사항 분석

3.1.1 기능적 요구사항

본 시스템은 Kubernetes 환경에서 발생하는 정책 위반 정보를 수집하고, 이를 개발자가 이해하기 쉬운 형태로 정리하여 제공해야 한다. 개발자는 대시보드를 통해 정책 위반 원인과 수정 방향을 확인할 수 있으며, 필요한 경우 정책 예외를 요청할 수 있어야 한다. 관리자는 예외 요청을 검토하여 승인 또는 반려할 수 있고, 승인된 예외는 실제 정책 예외로 반영되어야 한다.

또한 시스템은 예외의 유효기간과 만료 상태를 관리하고, 예외가 종료된 이후에도 요청 및 승인 이력을 보존해야 한다. 이를 통해 정책 예외가 일시적인 조치로 관리될 수 있도록 하며, 사후 감사와 운영 검토에 필요한 근거를 제공한다. 더불어 정책별 위반 빈도, 반복 위반, 만료 예정 예외 등 관리자의 의사결정에 필요한 거버넌스 정보를 대시보드 또는 외부 관측 도구를 통해 제공해야 한다.

3.1.2 비기능적 요구사항

본 시스템은 기존 Kubernetes 정책 엔진과 CI/CD 파이프라인을 대체하지 않고, 정책 위반 대응과 예외 관리를 보조하는 계층으로 동작해야 한다. 따라서 기존 배포 흐름에 미치는 영향을 최소화하면서도, 정책 위반 정보와 예외 처리 이력을 안정적으로 수집하고 관리할 수 있어야 한다.

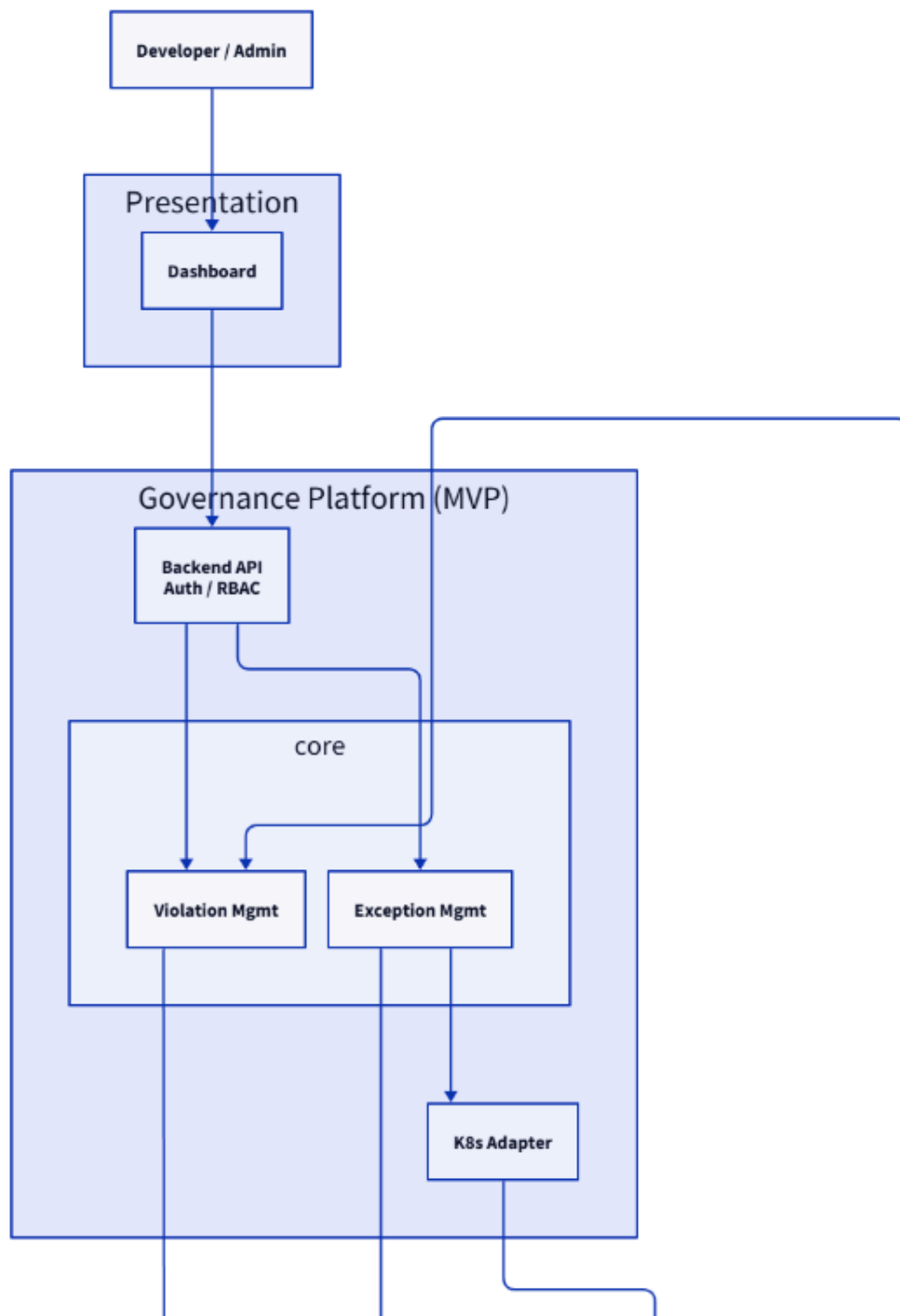
보안 측면에서는 일반 개발자와 관리자의 권한을 구분하고, 정책 예외 적용 권한을 제한된 범위로 관리해야 한다. 특히 예외는 보안 정책을 우회하는 기능이므로, 승인 절차와 적용 범위가 명확하게 통제되어야 한다. 또한 모든 예외 요청, 승인, 반려, 만료 이력은 감사 가능한 형태로 보존되어야 한다.

시스템은 초기에는 단일 Kubernetes 클러스터 환경을 기준으로 구현하되, 향후 멀티클러스터, 외부 알림, Prometheus/Grafana 연동, CI/CD 도구 연동 등으로 확장할 수 있는 구조를 가져야 한다. 또한 개발자와 관리자가 정책 위반 및 예외 상태를 쉽게 이해할 수 있도록 사용성을 고려해야 하며, 정책 집행 과정에 불필요한 성능 부담을 주지 않도록 설계되어야 한다.

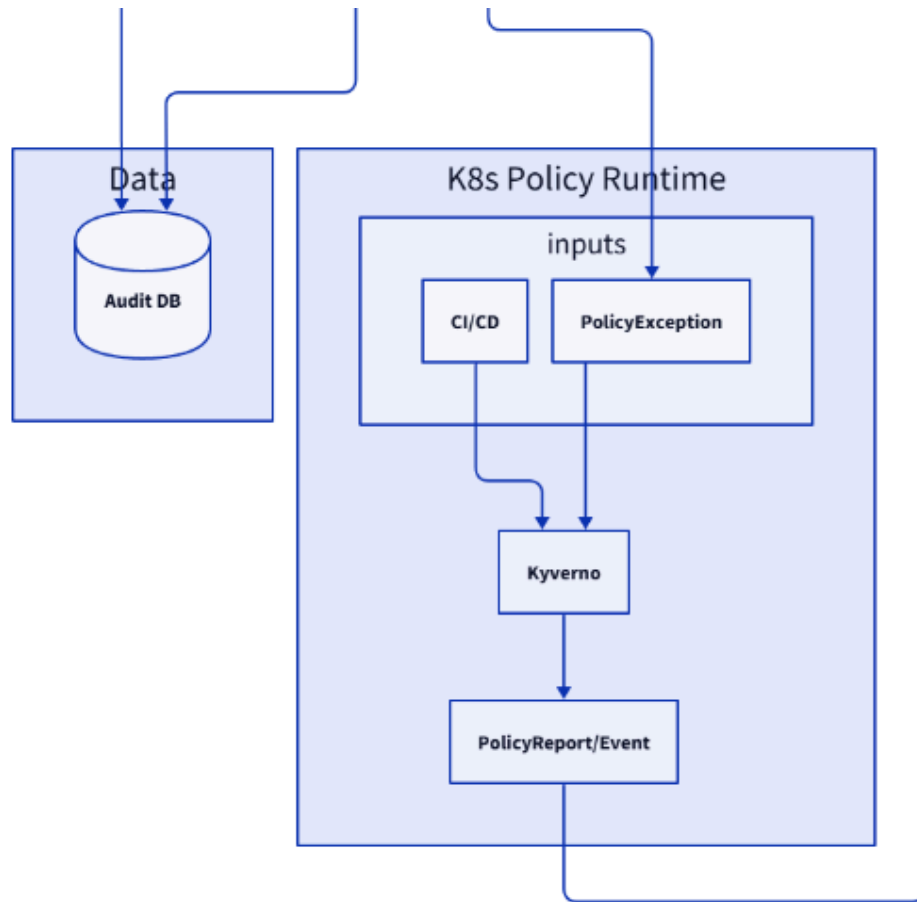
3.2 시스템 구성

본 프로젝트에서는 Kubernetes 환경의 정책 엔진으로 Kyverno를 대상으로 하여 시스템을 설계한다.

Kyverno는 Kubernetes-native 방식으로 정책을 정의할 수 있으며, PolicyException 리소스를 통해 특정 정책 또는 rule에 대한 예외를 표현할 수 있다. 따라서 정책 위반 피드백 개선과 예외 관리 자동화를 실험하기에 적합하다.

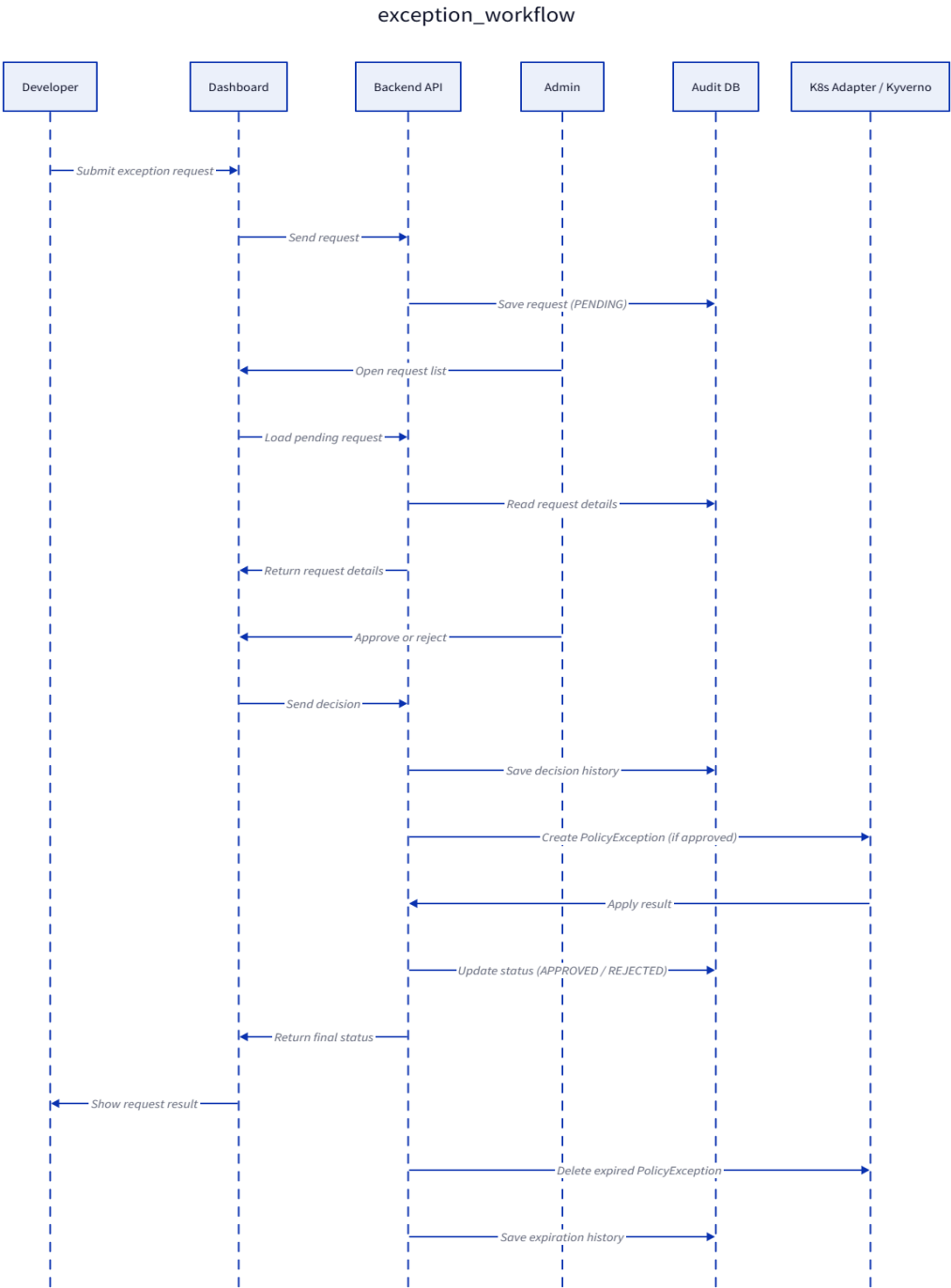


[그림 1 - 1] 시스템 구성도: 거버넌스 플랫폼 영역



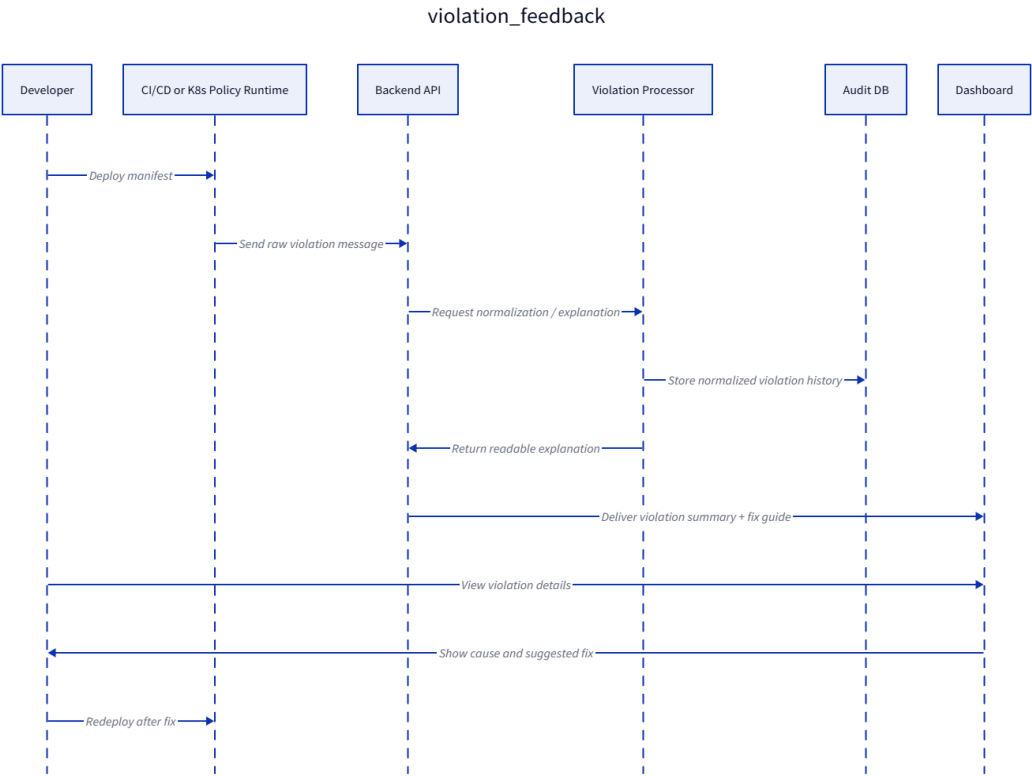
[그림 1 - 2] 시스템 구성도: 정책 런타임 및 데이터 계층

전체 구조는 그림 1-1과 그림 1-2와 같이 사용자와 상호작용하는 Presentation Layer, 핵심 업무 흐름을 처리하는 PaC Governance Platform, 실제 정책 평가가 수행되는 Kubernetes Policy Runtime, 그리고 감사 이력을 저장하는 Data Layer로 구분된다.



[그림 2] 정책엔진 예외 처리 시퀀스 다이어그램

그림 2는 개발자와 관리자 간의 예외 처리 흐름을 나타낸다. 개발자가 예외 요청을 하면 백엔드에서 예외 요청을 데이터베이스에 저장한다. 들어온 요청을 관리자가 승인하면 PolicyException 리소스를 생성하여, 다음 요청부터 해당 리소스를 통해 정책 예외가 이루어지는 방식으로 정책 예외 처리를 반영한다.



[그림 3] 정책 위반 피드백 제공 시퀀스 다이어그램

그림 3은 정책 위반 메시지를 사람이 이해할 수 있도록 변환하는 과정을 나타낸다. 정책 엔진에서 위반이 발생하면 백엔드에서 위반 결과를 전달받는다. 이후 전달받은 메시지를 정규화하고, 정책명과 rule명에 매핑된 설명 템플릿을 적용하여 개발자가 이해하기 쉬운 형태로 재구성한다. 이러한 처리가 완료되면 대시보드를 통해 개발자에게 가독성이 개선된 정책 위반 메시지를 보여준다.

4. 제약사항 및 기술적 어려움

4.1 오류 메시지 해석의 정확성

정책 위반 메시지를 사람이 이해하기 쉬운 형태로 재구성할 때, 원본 정책의 의도를 잘못 해석하면 오히려 혼란을 줄 수 있다. 따라서 자동 추론보다는 정책명과 rule명에 따른 템플릿 기반 설명을 우선한다.

이를 통해 설명의 일관성과 예측 가능성을 확보하고, 정책 관리자도 각 정책에 대한 설명 템플릿을 직접 관리할 수 있도록 한다.

4.2 관리자 인증 및 service account 권한 관리

본 프로젝트의 범위에서는 복잡한 기업용 SSO 전체 연동까지 구현하지 않는다. 대신 백엔드가 사용자의 권한을 확인하고, 관리자 권한이 확인된 경우에만 제한된 Kubernetes service account를 사용하여 PolicyException을 생성하는 구조로 설계한다.

해당 구현 요소는 기술적 난도가 있으나, 본 시스템이 정책 예외 생성 권한을 갖는다는 점에서 반드시 필요한 설계, 구현 요소이다. 필요한 리소스와 namespace에 대해서만 최소 권한을 부여하는 RBAC 설계가 중요하다.

4.3 만료 처리와 이력 보존의 분리

Kyverno cleanup 기능을 활용하면 만료된 PolicyException 리소스를 제거할 수 있다. 그러나 리소스가 삭제되면 클러스터의 현재 상태만으로는 과거 예외 이력을 확인하기 어렵다.

따라서 본 시스템은 cleanup 결과와 별도로 데이터베이스에 예외 요청, 승인, 반려, 생성, 만료 이력을 보존해야 한다. cleanup은 기술적 리소스 제거를 담당하고, 본 시스템은 예외 생명주기의 업무적·감사적 기록을 담당하는 구조로 역할을 분리한다.

4.4 데이터 저장 비용과 성능 문제

초기 구현은 싱글클러스터 기반이지만, 실제 운영 환경이나 향후 멀티클러스터 확장을 고려하면 정책 위반 이벤트와 예외 이력이 큰 볼륨으로 증가할 수 있다.

모든 이벤트를 동기적으로 데이터베이스에 저장하면 DB write 병목, 대시보드 조회 지연, 원본 로그 저장 비용 증가, 반복 이벤트로 인한 중복 데이터 증가 등의 문제가 발생할 수 있다. 또한 정책 집행 경로에 데이터베이스 작업이 직접 개입하면 admission 성능에 영향을 줄 수 있다.

따라서 본 시스템은 최초 설계부터 정책 집행 경로와 데이터 저장 경로를 분리하는 방향으로 설계한다. 정책 위반 수집과 통계 생성은 가능한 한 비동기적으로 처리하고, 원본 로그와 정규화 데이터, 감사 이력, 집계 데이터를 구분하여 관리한다.

학부 프로젝트에서 실제 대규모 부하를 검증하기는 어렵지만, 이러한 설계를 고려하는 것은 현업에 가까운 시스템 설계 경험으로서의 의미가 있다.

5.구현 계획 및 역할 분배

5.1 구현 계획

구분	추진 목표	5월	6월	7월	8월	9월
기획	요구사항 분석					
	시스템 구조 설계					
	DB 구현 및 더미 데이터 생성					
	Kubernetes 인프라 구축 및 CI/CD 파이프라인 연동					
	테스트케이스 설계					
서버 개발	사용자 인증/인가 API					
	Policy Engine API/Kubernetes 연동					
	정책 예외 처리 API 구현					
	정책 오류 관리 API 구현					
클라이언트 개발	대시보드 디자인					
	UI/로직 개발					
	UX 점검 및 개선					
	서버 API 연동					
테스트						
최종보고서 작성						

5.2 역할 분배

이름	역할
고영림	- 시스템 전체 구조 설계 - Kubernetes 기반 테스트 환경 구성 - CI/CD 파이프라인 구축 - 기술적 의사결정 정리 및 문서 작성
최유열	- 정책 위반/예외 처리 등 핵심 비즈니스 로직 개발 - DB를 활용한 API 구현

	- 관리자 승인 및 상태 관리 로직 구현 - 외부 시스템과의 연동 기능 구현
박태영	- 개발자와 관리자가 사용하는 대시보드 화면 구현 - 정책 위반 정보와 예외 요청 상태를 사용자 친화적으로 표현 - 승인·이력·운영 정보 조회 화면 구성